

GitHub Repository Analytics System

Kỹ thuật và Công nghệ Dữ liệu lớn

Lê Hồng Anh - 23020327
Hoàng Mạnh Hùng - 23020371
Đỗ Việt Dũng - 23020343

Trường Đại học Công nghệ - ĐHQGHN
Viện Trí tuệ Nhân tạo

Hà Nội, 2025

Nội dung trình bày

1. Tổng quan dự án
2. Kiến trúc & Công nghệ
3. Kết quả Phân tích Thực nghiệm
4. Tổng kết Demo

1. Tổng quan dự án

1.1 Bối cảnh & Lý do chọn đề tài

Tại sao chọn GitHub?

- Kho lưu trữ mã nguồn lớn nhất thế giới.
- Nơi diễn ra các xu hướng công nghệ mới nhất.
- Dữ liệu khổng lồ: Code, Commit, Issue, Star, Fork.

Nhu cầu thực tế:

- Nhà tuyển dụng cần biết công nghệ nào đang hot.
- Lập trình viên cần chọn công cụ, thư viện, framework phù hợp.



1.2 Những thách thức trong môi trường GitHub

- **Khối lượng dữ liệu lớn:** GitHub chứa hàng triệu repositories với dữ liệu liên tục tăng, yêu cầu hệ thống có khả năng mở rộng cao.
- **Độ phức tạp của dữ liệu:** Bao gồm metadata, README, issue, pull request... chứa nhiều dữ liệu bán cấu trúc hoặc không cấu trúc.
- **Tốc độ dữ liệu:** Các hoạt động diễn ra liên tục (real-time), đòi hỏi pipeline phân tích phải nhanh và hiệu quả.
- **Đảm bảo tính chính xác:** Dữ liệu từ nhiều nguồn không đồng nhất gây khó khăn cho thống kê và học máy.
- **Quản lý dữ liệu:** Lưu trữ, bảo mật và truy xuất dữ liệu trong môi trường phân tán gặp nhiều thách thức.
- **Chi phí tính toán:** Xử lý dữ liệu lớn đòi hỏi tài nguyên tính toán và lưu trữ đáng kể.
- **Tích hợp hệ thống:** Đồng bộ hóa nhiều công cụ (GitHub API, Spark, Redis, Flask) để giải quyết vấn đề tương thích.

1.3 Phạm vi dự án

Dự án bao gồm:

- **Thu thập & xử lý dữ liệu:** Thu thập repo, issue, commit từ GitHub API; làm sạch, chuẩn hóa và trích xuất đặc trưng.
- **Pipeline thời gian thực:** Dùng Spark Streaming xử lý trực tiếp, Redis lưu trữ kết quả và Flask hiển thị dashboard.
- **Ứng dụng học máy:** Dùng TF-IDF, LDA và similarity để phân tích chủ đề, mức độ phổ biến và liên kết.
- **Triển khai & trực quan hóa:** Tích hợp dashboard Flask, quan sát xu hướng repository, stars, forks theo thời gian.

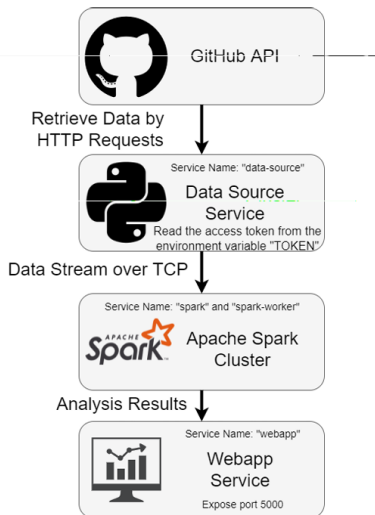
Dự án KHÔNG đi sâu vào:

- **Phân tích dài hạn:** Không phân tích dữ liệu lịch sử quá dài, tập trung vào xu hướng hiện tại.
- **Yếu tố bên ngoài:** Không xem xét các yếu tố kinh tế, xã hội hoặc chính trị.
- **Chất lượng mã nguồn:** Không kiểm tra chất lượng code chi tiết, chủ yếu dựa trên metadata.
- **Đa nền tảng:** Tập trung vào GitHub, không mở rộng sang GitLab hay Bitbucket.

2. Kiến trúc & Công nghệ

Kiến trúc tổng quan

2.4.1 Kiến trúc tổng thể



Hệ thống theo kiến trúc microservices 3 tầng, chạy trên Docker containers, xử lý dữ liệu streaming từ GitHub API.

- Tầng 1: Thu thập dữ liệu (Data Ingestion Layer)
- Tầng 2: Xử lý dữ liệu (Processing Layer): Apache Spark Cluster
- Tầng 3: Hiển thị kết quả: Web Application Service

2.1 Tổng quan hệ sinh thái công nghệ

Dự án tích hợp các công nghệ mã nguồn mở hàng đầu để xây dựng pipeline xử lý dữ liệu lớn gần thời gian thực (Near Real-time):

Xử lý trung tâm

Apache Spark Streaming

- Xử lý phân tán tốc độ cao (In-memory).
- Mô hình Micro-batch (DStreams).

Lưu trữ & Caching

Redis

- In-memory Key-Value Store.
- Giảm độ trễ truy xuất cho Dashboard.

Hạ tầng & Triển khai

Docker & Docker Compose

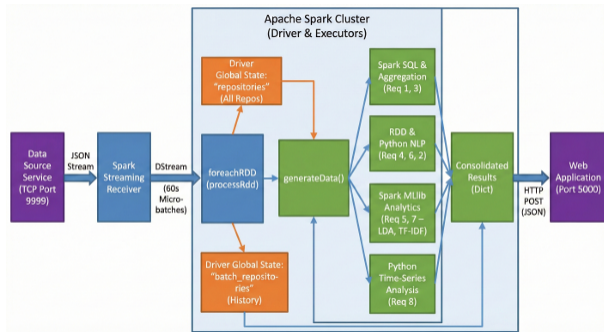
- Đóng gói môi trường đồng nhất.
- Điều phối đa dịch vụ (Multi-container).

Nguồn dữ liệu

GitHub REST API

- Cung cấp dữ liệu repository, metadata.
- Kết nối qua TCP Socket.

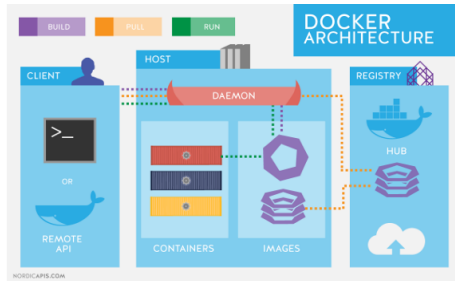
Chi tiết: Apache Spark Streaming



- **Vai trò:** Bộ xử lý trung tâm của hệ thống.
- **Cơ chế hoạt động:**
 - Chia dòng dữ liệu thành các **micro-batch** (60 giây).
 - Tận dụng RAM để xử lý nhanh hơn MapReduce truyền thống.
- **Các module sử dụng:**
 - **Spark Core/SQL:** Tổng hợp dữ liệu, tính toán thống kê (Yêu cầu 1, 3).
 - **Spark MLlib:** Chạy các thuật toán học máy như LDA, TF-IDF (Yêu cầu 5, 7).

Chi tiết: Docker Ecosystem & Redis

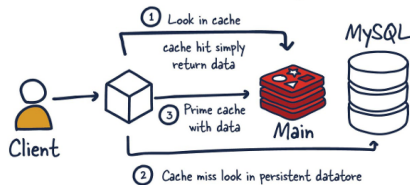
Docker Architecture



- **Isolation:** Mỗi service (Spark Master, Worker, Webapp) chạy trong container riêng biệt.
- **Scalability:** Dễ dàng mở rộng số lượng Spark Worker bằng lệnh `docker-compose scale`.

Redis (Caching Layer)

How is redis traditionally used



- **Vấn đề giải quyết:** Giảm tải cho Spark khi người dùng reload Dashboard liên tục.
- **Cơ chế:** Spark ghi kết quả JSON vào Redis → Flask đọc từ Redis → Frontend.

2.2.1 Thu thập dữ liệu (Ingestion)

Component: `data_source.py` (Python Service)

- **Nguồn:** GitHub API (qua thư viện `Yêu cầuuests`).
- **Giao thức truyền:** TCP Socket tại port 9999.
- **Định dạng:** NDJSON (Newline Delimited JSON).
- **Chiến lược:**
 - Chu kỳ gọi API: 15 giây/lần.
 - Mục tiêu: Python, Java, JavaScript.
 - Xác thực: GitHub Personal Access Token (PAT) để tăng Rate Limit.

Socket Streaming Code

```
1 # Gửi dữ liệu qua TCP
2 json_data = f"{json.dumps(data)}\n".encode()
3 conn.send(json_data)
```

2.2.2 Chiến lược Xử lý Streaming

Cấu hình Spark Context:

- **Batch Interval:** 60 giây (Cân bằng giữa độ trễ và khả năng xử lý NLP phức tạp).
- **Input:** `ssc.socketTextStream(IP, 9999)`.
- **Storage Level:** `MEMORY_AND_DISK` (Đảm bảo an toàn dữ liệu).

Quy trình trong `processRdd`:

- 1 **Ingest:** Nhận RDD từ DStream.
- 2 **Deduplication:** Loại bỏ bản tin trùng lặp.
- 3 **Update State:** Cập nhật biến toàn cục và lịch sử batch.
- 4 **Analytics:** Gọi 8 hàm phân tích chuyên biệt.
- 5 **Output:** Gửi kết quả tổng hợp qua HTTP POST.

Kỹ thuật: Khử trùng lặp & Quản lý trạng thái

Thách thức: API GitHub trả về các repo trùng lặp giữa các lần gọi liên tiếp.

Giải pháp 2 lớp:

- 1 **Batch Level:** Đảm bảo tính duy nhất trong 60s hiện tại.
- 2 **Global Level:** Duy trì Global Dictionary chứa tất cả ID đã xử lý.

Logic Deduplication (Python)

```
1 # Chỉ thêm nếu ID chưa từng xuất hiện trong Global
2 if repo['id'] not in repositories:
3     repositories[repo['id']] = repo
4     batch_repositories[repo['id']] = repo
```

2.3 Tổng quan 8 Nhiệm vụ phân tích

Hệ thống thực hiện 8 tác vụ phân tích song song trên mỗi micro-batch:

Nhóm Thống kê cơ bản:

- **Yêu cầu 1:** Tổng số Repo theo ngôn ngữ.
- **Yêu cầu 2:** Hoạt động Push trong 60s qua (Real-time).
- **Yêu cầu 3:** Số sao trung bình (Avg Stars).

Nhóm Xử lý Ngôn ngữ (NLP):

- **Yêu cầu 4:** Top 10 từ khóa phổ biến.
- **Yêu cầu 7:** Độ tương đồng văn bản (TF-IDF + Cosine Similarity).

Nhóm Phân tích Nâng cao (ML):

- **Yêu cầu 5:** Mô hình chủ đề (LDA - Latent Dirichlet Allocation).
- **Yêu cầu 6:** Nhận diện thực thể (NER - Công ty, Tool, Framework).

Nhóm Xu hướng:

- **Yêu cầu 8:** Phân tích chuỗi thời gian (Time-series Growth).

Chi tiết: NLP Pipeline & Topic Modeling

Tiền xử lý văn bản (Yêu cầu 4, 5, 7):

- Làm sạch (Regex) → Lowercase → Tokenization → Stop-words Removal.

Topic Modeling với LDA (Yêu cầu 5):

- Sử dụng **Spark MLlib**.
- **Input:** Vector TF-IDF của phần mô tả (Description).
- **Output:** 5 chủ đề tiềm ẩn (Hidden Topics) cho mỗi ngôn ngữ.
- **Optimizer:** `online` (Phù hợp cho dữ liệu streaming).

Named Entity Recognition (Yêu cầu 6):

- Phương pháp: Dictionary-based Matching.
- Phân loại: Companies (Google, Meta...), Frameworks (React, Spring...), Tools (Docker, K8s...), Tech (AI, ML).

Chi tiết: Similarity & Time Series

Yêu cầu 7: Độ tương đồng (Similarity)

- **Mục tiêu:** Tìm các dự án có nội dung tương tự.
- **Kỹ thuật:** TF-IDF Vectorization + Cosine Similarity.
- **Tối ưu:** Giới hạn so sánh 1-vs-5 (so sánh repo hiện tại với 5 repo lân cận) để giảm độ phức tạp tính toán $O(N^2)$.

Yêu cầu 8: Chuỗi thời gian (Time Series)

- **Bucket:** Gom nhóm theo tháng (YYYY-MM).
- **Chỉ số:** Tốc độ tăng trưởng (Growth Rate).
- **Phân loại xu hướng:**
 - Growing ($> 10\%$)
 - Declining ($< -10\%$)
 - Stable

2.4.1 Kiến trúc Microservices

Hệ thống được triển khai với 5 dịch vụ container hóa, quản lý bởi Docker Compose:

- ➊ **data-source:** Service Python thu thập dữ liệu từ GitHub API.
- ➋ **spark-master:** Quản lý tài nguyên cluster, điều phối job.
- ➌ **spark-worker:** Thực thi các task xử lý phân tán (1GB RAM/worker).
- ➍ **redis:** Cơ sở dữ liệu đệm (Cache).
- ➎ **webapp:** Flask Server phục vụ Dashboard & API REST.

2.4.2 Luồng dữ liệu triển khai (End-to-End)

[Sơ đồ luồng dữ liệu End-to-End]

Quy trình khép kín:

- 1 **Collect:** `data-source` gọi GitHub API (15s/lần).
- 2 **Stream:** Gửi JSON qua TCP socket (port 9999) tới Spark.
- 3 **Process:** Spark Streaming xử lý batch (60s), tính toán 8 metrics.
- 4 **Transmit:** Spark gửi kết quả tổng hợp (JSON) qua HTTP POST tới `webapp`.
- 5 **Cache:** `webapp` lưu dữ liệu mới nhất vào Redis.
- 6 **Visualize:** Frontend (JS) poll dữ liệu từ `webapp` để cập nhật biểu đồ.

Cấu hình Docker Compose & Mạng

Mạng nội bộ (Internal Network):

- Các service giao tiếp qua tên container (DNS Resolution).
- Ví dụ: Spark kết nối worker qua `spark://spark-master:7077`.

Quản lý Dependency:

- Cơ chế `depends_on` đảm bảo Redis và Spark Master khởi động trước khi các service phụ thuộc chạy.
- **Volumes:** Mount code từ máy host vào container để dễ dàng cập nhật logic mà không cần rebuild image liên tục.

Triển khai: Chỉ cần 1 lệnh duy nhất:

```
1 docker-compose up -d
```

3. Kết quả Phân tích Thực nghiệm

3.1 Yêu cầu 1: Tổng số Repository theo ngôn ngữ

Kết quả thực nghiệm:

- **Python:** 186 repositories.
- **Java:** 177 repositories.
- **JavaScript:** 50 repositories.

Nhận xét:

- Python chiếm ưu thế về số lượng thu thập được trong phiên chạy.
- Dữ liệu được tích lũy từ thời điểm bắt đầu Streaming, đảm bảo không trùng lặp (Deduplication).

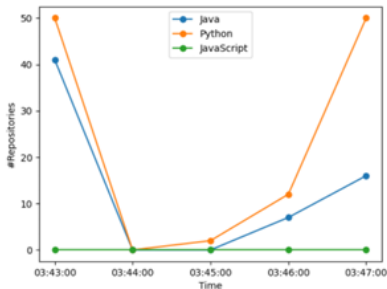
Total number of collected repositories

Python: 186
Java: 177
JavaScript: 50

Hình: Phân bố số lượng Repository

3.2 Yêu cầu 2: Hoạt động Push trong 60s gần nhất

Number of collected repositories with changes pushed during the last 60 seconds



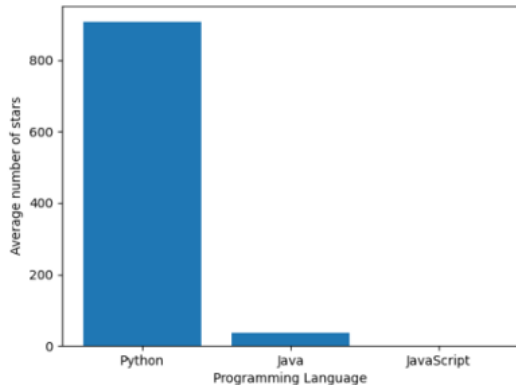
Hình: Biến động số lượng Push theo thời gian

Phân tích xu hướng:

- **Python:** Dao động mạnh, có thời điểm giảm xuống 0 rồi tăng lại lên 50 repo/batch.
- **Java:** Xu hướng tương tự, hồi phục chậm hơn Python.
- **JavaScript:** Không ghi nhận hoạt động push trong khoảng thời gian đo đạc.

3.3 Yêu cầu 3: Số sao trung bình (Avg Stars)

Average number of stars by language



Thống kê độ phổ biến:

- **Python:** $\approx 880 - 890$ sao/repo (Cao nhất).
- **Java:** $\approx 40 - 50$ sao/repo.
- **JavaScript:** Thấp hoặc chưa đủ dữ liệu thống kê trong phiên.

Ý nghĩa:

- Các dự án Python thu thập được có mức độ quan tâm từ cộng đồng rất lớn.

3.4 Yêu cầu 4: Top 10 Từ khóa phổ biến (NLP)

Total 10 most frequent words

Java

and, 8
a, 7
is, 5
the, 5
for, 4
de, 4
with, 4
to, 4
using, 4
this, 3

JavaScript

and, 7
for, 5
with, 5
a, 5
web, 3
application, 3
react, 3
libros, 3
script, 3
tracking, 3

Kết quả NLP:

- **Stop-words:** "and", "for", "the" xuất hiện nhiều nhất ở mọi ngôn ngữ.
- **JavaScript:** Nổi bật các từ khóa web: "react", "web", "application".

3.5 Yêu cầu 5: Phân tích chủ đề (LDA Topic Modeling)

Python - Emerging Topics

Topic 1: [mock](#), [testing](#), [repository](#)

Topic 2: [de](#), [simple](#), [pour](#)

Topic 3: [data](#), [&](#), [daily](#)

Topic 4: [new](#), [+](#), [crypto](#)

Topic 5: [using](#), [repo](#), [first](#)

Java - Emerging Topics

Topic 1: [platform](#), [플랫폼](#), [development](#)

Topic 2: [mock](#), [testing](#), [repository](#)

Topic 3: [spring](#), [de](#), [backend](#)

Topic 4: [created](#), [libraries](#), [judge](#)

Topic 5: [system](#), [daily](#), [tool](#)

JavaScript - Emerging Topics

Topic 1: [app](#), [de](#), [quality](#)

Topic 2: [y](#), [personal](#), [para](#)

Topic 3: [using](#), [built](#), [react](#)

Topic 4: [testing](#), [mock](#), [javascript](#)

Topic 5: [.](#), [project](#), [ai](#)

Những chủ đề này cho thấy pipeline ingestion có thể phát hiện nhiều miền—kiểm thử, backend, nền tảng đa ngôn ngữ và demo AI—ở cả ba ngôn ngữ.

3.6 Yêu cầu 6: Hệ sinh thái công nghệ (NER)

Named Entity Recognition - Technology Ecosystem

Python - Technology Ecosystem

Companies:

github (7) intel (2) nvidia (1) apple (1) docker (1)

Frameworks:

pytorch (1) vue (1) flask (1) express (1) react (1)

Technologies:

ar (26) ai (17) api (8) web (4) rest (2)

Tools:

git (9) mysql (1) docker (1)

Java - Technology Ecosystem

Companies:

github (2) uber (1) gitlab (1)

Frameworks:

spring (2) react (1)

Technologies:

ar (14) ai (4) microservices (2) cloud (2) api (1)

Tools:

git (3)

JavaScript - Technology Ecosystem

Companies:

github (1)

Frameworks:

react (3) express (3) spark (1)

Technologies:

ar (17) web (9) ai (5) api (2) rest (1)

Tools:

git (4) mongodb (1)

Điểm nhấn Hệ sinh thái:

- **Công nghệ hot:** "AR" (Augmented Reality) và "AI" xuất hiện dày đặc ở cả 3 ngôn ngữ.
- **Python:** Hệ sinh thái đa dạng nhất (pytorch, flask, nvidia, intel).
- **JavaScript:** Thiên về Web (react, express, mongodb).
- **Java:** Thiên về Enterprise (spring, microservices).
- **Tool:** "git", "github" xuất hiện phổ biến.

3.7, Yêu cầu 7: Tương đồng văn bản (TF-IDF Similarity)

TF-IDF Similarity - Related Projects

Python - Similar Repository Pairs

Analyzed 21 repositories

No similar repository pairs found

Java - Similar Repository Pairs

Analyzed 18 repositories

Similarity: 0.153
Sorveteria-Delicia
Projeto de Pesquisa de Satisfação de uma Sorveteri...

DAWBIO_EXERCICIS_ALUMNES
Aqui hi haura les diferents entregues de deures de...

Similarity: 0.139
AtividadeContinuadaPOO
Este repositorio contem minha stividade continuada...

Sorveteria-Delicia
Projeto de Pesquisa de Satisfação de uma Sorveteri...

JavaScript - Similar Repository Pairs

Analyzed 23 repositories

Similarity: 0.235
softuni-bookstore-nov-2025
SoftUni ReactJS Course Project...

ditacourse
Content for Course on DITA...

Similarity: 0.126
sabores-do-norte-v2
Versão 2.0 do restaurante Sabores do Norte — front...

Printify-Frontend
Frontend of Printify...

Similarity: 0.123
ditacourse
Content for Course on DITA...

Events-Platform
A Platform that allows an administrator to set up ...

Phân tích độ tương đồng cosine TF-IDF tạo ra các cặp repo cụ thể:
Python 21 repo được phân tích; không repo nào vượt ngưỡng tương đồng (>0.1), nên không có cặp nào.
Những kết quả này cho thấy pipeline có khả năng phát hiện các repo hướng coursework trong Java và frontend giáo dục/thương mại trong JavaScript, trong khi corpus Python vẫn quá đa dạng để tìm ra cặp tương đồng mạnh.

3.8 Yêu cầu 8: Phân tích xu hướng (Time Series Analysis)



Hình: Thống kê xu hướng từ tháng 06/2025 - 11/2025

Kết quả phân tích dữ liệu:

- **Xu hướng chung:** Cả 3 ngôn ngữ (Python, Java, JavaScript) đều duy trì trạng thái **ỔN ĐỊNH (STABLE)** trong 6 tháng qua.
- **Về quy mô:** Python chiếm thị phần lớn nhất (~84 repos), tiếp theo là JavaScript (~60) và Java (~28).
- **Biến động:** Tỷ lệ thay đổi (Growth Rate) rất thấp, đa số các tháng là 0%, cho thấy nguồn dữ liệu ổn định.
- **Thời điểm đạt đỉnh:** Python đạt đỉnh vào tháng 10 (85 repos), trong khi Java và JS đạt đỉnh sớm hơn vào tháng 9.

4. Tổng kết Demo

Tổng kết Đánh giá

Thành tựu

- Ứng dụng được các kỹ thuật nâng cao (ML/NLP) trên dữ liệu dòng.
- Hệ thống có khả năng chịu lỗi và mở rộng tốt.
- Sinh biểu đồ chuỗi thời gian và bản đồ hệ sinh thái công nghệ.

Hạn chế

- Khoảng xử lý 60 giây chưa đạt mức sub-second latency.
- Quản lý trạng thái bằng Memory tốn tài nguyên RAM khi chạy lâu dài.
- Chỉ có một Spark worker nên khả năng xử lý song song còn hạn chế.

Hướng phát triển trong tương lai

1 Nâng cấp Hạ tầng:

- Tích hợp **Apache Kafka** làm Message Queue.
- Triển khai trên Kubernetes (K8s) để auto-scaling.

2 Cải thiện Thuật toán:

- Dùng **Incremental LDA** để cập nhật chủ đề liên tục.
- Áp dụng Deep Learning (LSTM) để dự báo xu hướng.

DEMO & TRẢ LỜI CÂU HỎI